LESSON 19

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Tilt Maze

Feel gravity with a motion sensor and build a tilt-to-play game.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- What an **accelerometer** is and how it feels gravity
- How to talk to a chip over **I²C** with the `Wire` library
- How to make a **negative number** out of two bytes
- How to build a game: moving sprites, **collisions** and a score

SESSION PLAN

0:00	How does a phone know which way is up?
0:10	Wire the GY-6500 and the OLED
0:25	Sketch 1: **L19_Tilt** + Serial Plotter
0:50	Short break
0:55	Sketch 2: **L19_RollingBall**
1:15	Sketch 3: **L19_DotGame** (5 changes)
1:35	Challenges: build a maze
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable, 2 breadboards
- ☐ GY-6500 motion sensor module
- ☐ 0.96" OLED display
- ☐ Passive buzzer
- ☐ Jumper wires

1. Feeling gravity

When you turn your phone sideways, the screen rotates. How does the phone know? It has an **accelerometer** inside: a tiny chip with a microscopic weight on springs. When you tilt it, gravity pulls the weight to one side, and the chip measures how far.

Your kit has one too: the **GY-6500** module. It measures in 3 directions, called **axes**: **x** (left ↔ right), **y** (forward ↔ back) and **z** (up ↕ down).

> **Big idea: 1 g = 16384**
>
> Lying flat, gravity pulls straight down, so **z** reads about **16384**. That number means "1 g", the pull of Earth's gravity. Tilt the sensor and some of that pull moves to x or y. The numbers can be **negative** too: tilting left gives a negative x, tilting right a positive x.

2. Wire it up

The GY-6500 and the OLED both use **I²C**, a way for many chips to share the same two wires, **SDA** (data) and **SCL** (clock). Each chip has its own **address**: the OLED is `0x3C`, the GY-6500 is `0x68`. So you connect both to the same pins!

Sharing wires

Put the two SDA wires in the same breadboard row as ESP32 pin 21, and the two SCL wires in the same row as pin 22.

✓	From	To
☐	GY-6500 **VCC**	**3V3**
☐	GY-6500 **GND**	**GND**
☐	GY-6500 **SDA**	ESP32 pin **21**
☐	GY-6500 **SCL**	ESP32 pin **22**
☐	OLED **VCC**, **GND**	**3V3**, **GND**
☐	OLED **SDA**, **SCL**	pins **21**, **22** (same rows as above)
☐	Buzzer **+** / **–**	pin **33** / **GND**

Leave the other GY-6500 pins (EDA, ECL, AD0, INT, NCS, FSYNC) empty. Push the GY-6500 into the breadboard **flat and level**: it's going to be your game controller!

3. Sketch 1: Read the tilt

Type this in and save as **L19_Tilt**. Upload, then open **Tools** → **Serial Plotter** (115200 baud) and tilt your breadboard.

What's a register?

Think of the chip as a row of numbered lockers, called **registers**. Some lockers hold settings (like locker 0x6B, the on/off switch). Others hold measurements (lockers 0x3B to 0x40 hold the tilt). We read and write lockers by their number.

Type this in → save it as **L19_Tilt** 45 lines

```
// Lesson 19: Read the tilt sensor
// GY-6500 on I2C (SDA 21, SCL 22). Open the Serial Plotter!

#include <Wire.h>

const int IMU = 0x68;
int ax, ay, az;

int readAxis() {
  int high = Wire.read();
  int low = Wire.read();
  int value = high * 256 + low;
  if (value > 32767) value = value - 65536;
  return value;
}

void readAccel() {
```

```
  Wire.beginTransmission(IMU);
  Wire.write(0x3B);               // first accelerometer register
  Wire.endTransmission(false);
  Wire.requestFrom(IMU, 6);
  ax = readAxis();
  ay = readAxis();
  az = readAxis();
}

void setup() {
  Serial.begin(115200);
  Wire.begin();
  Wire.beginTransmission(IMU);
  Wire.write(0x6B);               // power register
  Wire.write(0);                  // 0 = wake up!
  Wire.endTransmission();
}

void loop() {
  readAccel();
  Serial.print("x:");
  Serial.print(ax);
  Serial.print(" y:");
  Serial.print(ay);
  Serial.print(" z:");
  Serial.println(az);
  delay(100);
}
```

How it works

Code	What it means
`Wire.begin();`	Start I²C on pins 21 (SDA) and 22 (SCL)
`Wire.beginTransmission(IMU);` … `Wire.endTransmission();`	Send a message to the chip at address `0x68`
`Wire.write(0x6B); Wire.write(0);`	"Set **register** 0x6B to 0." The chip starts asleep; this wakes it up
`Wire.write(0x3B);`	"I want to read, starting at register 0x3B" (where x is stored)
`Wire.requestFrom(IMU, 6);`	Ask for 6 bytes: x, y and z, two bytes each
`high * 256 + low`	Glue two bytes (0–255) into one number (0–65535)
`if (value > 32767) value = value - 65536;`	Numbers over half-way are really **negative**. This is called **two's complement**

Try it

1. Lay it flat. What does **z** say? Turn the breadboard upside down. What now?
2. Which way do you tilt to make **x** go positive? And **y**?
3. Shake it! What happens to the lines?

4. Sketch 2: Rolling ball

Save as **L19_RollingBall** and type this in. Upload it and tilt the breadboard to roll the ball.

Type this in → save it as **L19_RollingBall** — 53 lines

```
// Lesson 19: Rolling ball
// Tilt the breadboard to roll a ball around the OLED screen.

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

Adafruit_SSD1306 display(128, 64, &Wire, -1);

const int IMU = 0x68;
int ax, ay;
float ballX = 64;
float ballY = 32;

int readAxis() {
  int high = Wire.read();
  int low = Wire.read();
  int value = high * 256 + low;
  if (value > 32767) value = value - 65536;
  return value;
}

void readAccel() {
  Wire.beginTransmission(IMU);
  Wire.write(0x3B);
  Wire.endTransmission(false);
  Wire.requestFrom(IMU, 4);
  ax = readAxis();
  ay = readAxis();
}

void setup() {
  Wire.begin();
  Wire.beginTransmission(IMU);
  Wire.write(0x6B);
  Wire.write(0);
  Wire.endTransmission();
```

```
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
}

void loop() {
  readAccel();
  ballX = ballX - ax / 2000.0;   // wrong way? change - to +
  ballY = ballY + ay / 2000.0;
  ballX = constrain(ballX, 3, 124);
  ballY = constrain(ballY, 3, 60);

  display.clearDisplay();
  display.drawRect(0, 0, 128, 64, WHITE);
  display.fillCircle(ballX, ballY, 3, WHITE);
  display.display();
  delay(20);
}
```

We only need x and y now, so we ask for **4** bytes instead of 6.

Code	What it means
`float ballX = 64;`	A **float** can hold decimals like 64.37, so the ball moves smoothly
`ballX = ballX - ax / 2000.0;`	Tilt more and the ball moves further each frame. `2000.0` controls the speed
`constrain(ballX, 3, 124)`	Keep the ball inside the screen
`fillCircle(x, y, 3, WHITE)`	Draw a filled circle with radius 3

Ball rolls the wrong way? It depends how your sensor sits. Change the `-` to a `+` (or the other way) on the `ballX` or `ballY` line.

5. Sketch 3: Collect the dots

Now let's turn it into a game. You don't need to type it all again! Use **File → Save As…** to save **L19_RollingBall** as **L19_DotGame**, then make these **5 changes**. New lines are shown in the boxes.

> **Big idea: collision detection**
>
> Every video game checks "are these two things touching?" many times a second. It's called **collision detection**. Here we check if the gaps in x and y are both small.

Change 1: under `float ballY = 32;` add:

```
const int BUZZER_PIN = 33;
int dotX, dotY;
int score = 0;
```

Change 2: above `void setup()` add a new function:

```
void newDot() {
  dotX = random(8, 120);
  dotY = random(12, 56);
}
```

Change 3: at the end of `setup()`, under `display.begin(…);` add:

```
  display.setTextColor(WHITE);
  newDot();
```

Change 4: in `loop()`, under `ballY = constrain(ballY, 3, 60);` add:

```
  if (abs(ballX - dotX) < 5 && abs(ballY - dotY) < 5) {
    score = score + 1;
    tone(BUZZER_PIN, 1500, 60);
    newDot();
  }
```

Change 5: under `display.drawRect(0, 0, 128, 64, WHITE);` add:

```
  display.setCursor(4, 4);
  display.print(score);
  display.drawCircle(dotX, dotY, 2, WHITE);
```

Upload and play! Stuck? Check each change is in the right place, inside the right `{ }`.

Code	What it means
`random(8, 120)`	A random whole number from 8 up to 119
`abs(ballX - dotX) < 5`	`abs` removes the minus sign. True if the ball and dot are less than 5 pixels apart
`&&`	**and**: both must be true. The ball must be close in x **and** y

6. Challenges

Challenge 1: Speed run

Add a 30-second timer using `millis()`. When time is up, show `GAME OVER` and the score, wait 3 seconds, and restart.

Challenge 2: Danger dot

Add a second dot (draw it as a square) that **loses** a point if you touch it. Make it move to a new place every 3 seconds.

Challenge 3: Tilt maze

Draw walls with `display.fillRect(x, y, w, h, WHITE)`. Stop the ball moving into a wall, and give the player a goal square to reach. Show how many seconds it took!

Bonus: Spirit level

Make a level for hanging pictures: show "LEVEL" on the screen and beep when `ax` is between -300 and 300.

7. Quiz

Check what you learned

1. Lying flat on the table, which axis feels gravity?

2. Why can the OLED and GY-6500 share the same two wires?

3. What does `&&` mean in an `if`?

4. Name a game or device that uses an accelerometer.

Tidy up: unplug the USB cable. Leave the OLED wired; you'll use it again in your final project.